ফ্রি চিট শিট

ইফতে সামউল ওহী
iftesamulohy.com

# Git ও GitHub চিট শিট

প্রতিদিনের ১০০-র বেশি command, ভুল শোধরানোর ছক আর সাধারণ error-এর সমাধান — সব এক জায়গায়, সহজ বাংলায়। প্রিন্ট করে ডেস্কের পাশে রাখুন।

১ Setup আর প্রতিদিনের কাজ

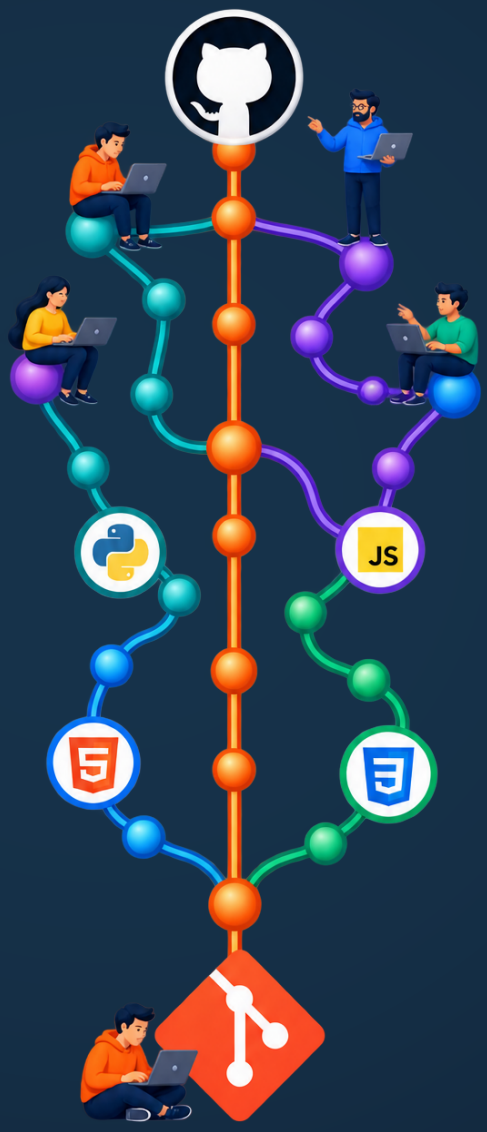
২ History আর ভুল শোধরানো

৩ Branch, merge, rebase, tag

৪ GitHub: push, pull, fork, PR

৫ কোন ভুলে কোন command — ছক

৬ সাধারণ error আর সমাধান




**পুরো ব্যাখ্যা চাই?**
এই চিট শিটের প্রতিটা command এসেছে ২৯ অধ্যায়ের বই "Git ও GitHub: আদি থেকে অন্ত" থেকে।

বইটি দেখুন →

Licensed to sakilorb@gmail.com · Do not redistribute

# Setup ও প্রতিদিনের কাজ

⚠ চিহ্ন = commit না করা কাজ মুছে যেতে পারে

## প্রথম দিনের setup — অধ্যায় ৩

| | |
|---|---|
| `git --version` | Git আছে কিনা, কোন version |
| `git config --global user.name "নাম"` | নিজের নাম (space থাকলে "..." জরুরি) |
| `git config --global user.email "email"` | GitHub-এর email বা noreply email |
| `git config --global init.defaultBranch main` | নতুন repo-তে `main` |
| `git config --global core.editor "code --wait"` | Editor হিসেবে VS Code |
| `git config --global pull.rebase true` | Pull-এ merge commit-এর জঞ্জাল না |
| `git config --global push.autoSetupRemote true` | নতুন branch-এর প্রথম push সহজ |
| `git config --global --list` | সব setting দেখা |

## শুরু করা — অধ্যায় ৪, ১৯

| | |
|---|---|
| `git init` | এই ফোল্ডারকে repo বানাও |
| `git clone <ঠিকানা>` | GitHub থেকে পুরো repo, history সহ |
| `git clone <ঠিকানা> নতুন-নাম` | অন্য ফোল্ডার-নামে |
| `git status` | এখন কী অবস্থা — সন্দেহ হলেই |
| `git status -s` | ছোট রূপে (`?? A M D`) |
| `git status -sb` | সাথে GitHub-এর চেয়ে কত এগিয়ে/পিছিয়ে |

## Stage আর commit — অধ্যায় ৫, ৬

| | |
|---|---|
| `git add ফাইল` | একটা ফাইল stage |
| `git add .` | এই ফোল্ডার থেকে নিচে সব |
| `git add -A` | পুরো repo-র সব পরিবর্তন |
| `git add -p` | ফাইলের অংশ বেছে বেছে (y / n / s / q) |
| `git commit -m "message"` | Commit |
| `git commit -m "শিরোনাম" -m "কেন"` | শিরোনাম + বিস্তারিত |
| `git commit -am "message"` | চেনা ফাইল stage+commit (নতুন ফাইল নেয় না) |
| `git commit --amend` | শেষ commit ঠিক করা (push-এর আগে) |
| `git commit --amend --no-edit` | বাদ পড়া ফাইল শেষ commit-এ যোগ |

## পরিবর্তন দেখা — অধ্যায় ৭

| | |
|---|---|
| `git diff` | যা বদলেছেন, stage করেননি |
| `git diff --staged` | পরের commit-এ যা যাবে — commit-এর আগে দেখুন |
| `git diff HEAD` | শেষ commit-এর পরে সব |
| `git diff --stat` | শুধু কোন ফাইলে কত লাইন |
| `git diff --cached --check` | ভুলে রয়ে যাওয়া conflict marker খোঁজা |

Diff পড়া: `-` লাইন মুছেছে, `+` লাইন এসেছে, বাকিটা প্রসঙ্গ।

## তিনটা এলাকা — অধ্যায় ৫

**Working dir** আপনার ফাইল → `git add` → **Staging** পরের commit-এ যা যাবে → `git commit` → **Repository** সব history

`status -s`-এর বাম কলাম = staging, ডান কলাম = আপনার ফাইল। `MM` মানে add-এর পরে আবার বদলেছেন — আবার `git add` দিন।

## ভালো commit message — অধ্যায় ৬, ২৪

| | |
|---|---|
| ✘ `update`, `fix`, `final` | ✔ পাঞ্জাবির দাম ১২০০ টাকা করা |
| শিরোনাম ~৫০ অক্ষর, বলবে কী করে | Body-তে বলবে **কেন** |
| একটা commit = একটা কাজ | `feat:` `fix:` `docs:` `chore:` |

## প্রতিটা commit-এর আগে তিন সেকেন্ড — অধ্যায় ৬.৬

**git status** → **git diff --staged** → **git commit -m "..."**

ধরা পড়ে ভুলে রয়ে যাওয়া পাসওয়ার্ড `.env`, `console.log` আর অর্ধেক লেখা কোড।


**শুধু command মুখস্থ করলে Git শেখা হয় না।**

বইয়ে একটা দোকানের ওয়েবসাইট বানাতে বানাতে প্রতিটা command কেন, কখন আর কীভাবে — আসল output সহ।

বইটি দেখুন →





Licensed to saikhan@gmail.com · Do not redistribute

# History ও ভুল শোধরানো

যা commit হয়েছে, তা প্রায় কখনো হারায় না

## History দেখা

অধ্যায় ৭

| | |
|---|---|
| `git log --oneline` | এক লাইনে এক commit (বের হতে q) |
| `git log --oneline -5` | শুধু শেষ ৫টা |
| `git log --stat` | প্রতিটা commit-এ কোন ফাইল |
| `git log -p` | প্রতিটা commit-এর পুরো diff |
| `git log --oneline -- ফাইল` | একটা ফাইলের history |
| `git log --grep="শব্দ"` | Message-এ খোঁজা |
| `git log --author="নাম"` | কারো commit |
| `git log --since="2 weeks ago"` | নির্দিষ্ট সময় থেকে |
| `git log --oneline --graph --all --decorate` | সব branch-এর ছবি |
| `git show <hash>` | একটা commit-এর ভেতরে |
| `git show HEAD~3:ফাইল` | ফাইলটা ৩ commit আগে কেমন ছিল |
| `git switch --detach <hash>` | পুরো প্রজেক্ট পুরনো অবস্থায় দেখা |
| `git switch main` | আবার আজকের অবস্থায় |

`HEAD` = এখন যেখানে · `HEAD~1` = এক commit আগে · `HEAD~2` = দুই commit আগে

## Commit-এর আগের ভুল

অধ্যায় ৯

| | |
|---|---|
| `git restore ফাইল` | ফাইলের বদল বাতিল ⚠ |
| `git restore .` | সব ফাইলের বদল বাতিল ⚠ |
| `git restore -p ফাইল` | শুধু একটা অংশ বাতিল |
| `git restore --staged ফাইল` | Staging থেকে নামানো, কাজ নিরাপদ |
| `git restore --source=HEAD~2 ফাইল` | পুরনো commit থেকে ফাইল আনা |
| `git clean -nd` | কোন untracked ফাইল মুছবে — শুধু দেখায় |
| `git clean -fd` | Untracked ফাইল-ফোল্ডার মোছা ⚠ |

## Commit-এর পরের ভুল

অধ্যায় ১০

| | |
|---|---|
| `git reset --soft HEAD~1` | শেষ commit ভাঙো, কাজ staged থাকুক |
| `git reset HEAD~1` | শেষ commit ভাঙো, কাজ unstaged থাকুক |
| `git reset --hard HEAD~1` | শেষ commit আর কাজ দুটোই ফেলো ⚠ |
| `git revert <hash>` | উল্টো commit — push করা commit-এর জন্য |
| `git revert HEAD~2..HEAD` | শেষ দুটো commit উল্টানো |
| `git revert -m 1 <merge-hash>` | একটা merge বাতিল |

**সোনালি নিয়ম:** push করা commit-এ `reset`/`amend` না, `revert`।

## Stash: কাজ সরিয়ে রাখা

অধ্যায় ১১

| | |
|---|---|
| `git stash push -u -m "নাম"` | নতুন ফাইলসহ, নাম দিয়ে সরাও |
| `git stash list` | তাকে কী কী আছে |
| `git stash show -p stash@{1}` | ভেতরে কী আছে |
| `git stash pop` | ফেরত আনো, তাক থেকে মোছো |
| `git stash apply stash@{1}` | ফেরত আনো, তাকেও রাখো |
| `git stash drop stash@{1}` | একটা মোছা |
| `git stash branch নাম` | Stash থেকে নতুন branch |

## সব হারিয়ে ফেলেছি?

অধ্যায় ১২

| | |
|---|---|
| `git reflog` | HEAD-এর চলাফেরার পুরো খাতা |
| `git reset --hard <hash>` | হারানো commit-এ ফেরা |
| `git branch উদ্ধার <hash>` | মুছে যাওয়া branch ফেরত |
| `git reset --hard ORIG_HEAD` | ঠিক আগের rebase/merge বাতিল |
| `git fsck --lost-found` | শুধু add করা কাজের শেষ আশা |

Commit না করা, add-ও না করা কাজ কোনো উপায়েই ফেরে না। তাই ঘনঘন commit।


Licensed to sakiron@gmail.com · Do not redistribute

# Branch, merge ও GitHub

push = পাঠাও · fetch = দেখো · pull = আনো ও মেলাও

## Branch — অধ্যায় ১৩

| | |
|---|---|
| `git branch -v` | সব branch, শেষ commit সহ |
| `git switch -c fix/cart-total` | নতুন branch বানিয়ে যাও |
| `git switch নাম` | Branch-এ যাও |
| `git switch -` | আগের branch-এ ফেরো |
| `git branch -m নতুন-নাম` | নাম বদল |
| `git branch --merged` | কোনগুলো মেলানো শেষ |
| `git branch -d নাম` | মেলানো branch মোছা |
| `git branch -D নাম` | না মেলানো branch জোর করে মোছা ⚠ |

নাম: `feature/…` `fix/…` `hotfix/…` `docs/…` — space চলে না, শুধু `feature` নামে branch বানাবেন না।

## Merge ও conflict — অধ্যায় ১৪, ১৫

| | |
|---|---|
| `git switch main`<br>`git merge feature` | আগে গন্তব্যে যান, তারপর উৎসের নাম |
| `git merge --no-ff feature` | সবসময় merge commit |
| `git merge --abort` | Conflict দেখে পিছিয়ে আসা |
| `git add ফাইল → git commit` | Conflict মিটিয়ে merge শেষ |
| `git restore --ours ফাইল` | পুরো ফাইলে আমার দিক |
| `git restore --theirs ফাইল` | পুরো ফাইলে ওদের দিক |
| `git config --global merge.conflictStyle diff3` | Conflict-এ মূল লেখাও দেখাও |

`<<<<<<< HEAD` আমার দিক · `=======` দেয়াল · `>>>>>>> feature` যেটা আনছি। চিহ্নগুলো মুছে ফাইল যেমন হওয়া উচিত তেমন লিখুন।

## Rebase ও tag — অধ্যায় ১৬, ১৭

| | |
|---|---|
| `git rebase main` | নিজের branch `main`-এর উপর বসাও |
| `git rebase --continue` | Conflict মিটিয়ে `git add`-এর পরে |
| `git rebase --abort` | সব বাতিল |
| `git rebase -i HEAD~4` | শেষ ৪টা commit গোছানো (pick / reword / squash / fixup / drop) |
| `git tag -a v1.0.0 -m "..."` | Release চিহ্ন |
| `git tag -n` | সব tag, message সহ |
| `git describe` | সবচেয়ে কাছের version থেকে কত দূরে |
| `git cherry-pick <hash>` | একটা commit এখানে কপি |

শেয়ার করা branch-এ কখনো rebase না। SemVer: `MAJOR.MINOR.PATCH` — বাগ ঠিক → PATCH, নতুন ফিচার → MINOR।

## Remote: push, pull, fetch — অধ্যায় ১৯

| | |
|---|---|
| `git remote -v` | Remote-এর তালিকা |
| `git remote add origin <ঠিকানা>` | GitHub repo যোগ |
| `git remote set-url origin <ঠিকানা>` | ঠিকানা বদল (HTTPS ↔ SSH) |
| `git push -u origin main` | প্রথম push, upstream সহ |
| `git push` | নিজের commit পাঠাও |
| `git fetch` | নতুন কী আছে আনো, মেলাবে না |
| `git log main..origin/main` | Pull করলে কোন commit আসবে |
| `git pull` | আনো আর মেলাও |
| `git pull --rebase --autostash` | অর্ধেক কাজ থাকা অবস্থায় pull |
| `git push origin --delete নাম` | GitHub-এর branch মোছা |
| `git fetch --prune` | মুছে যাওয়া remote branch পরিষ্কার |
| `git push origin --tags` | Tag পাঠানো (সাধারণ push-এ যায় না) |
| `git push --force-with-lease` | শুধু নিজের branch-এ, rebase-এর পরে ⚠ |

## Fork ও Pull Request — অধ্যায় ২২

| | |
|---|---|
| `git remote add upstream <মূল-repo>` | Fork-এ মূল repo যোগ |
| `git fetch upstream` | মূল repo-র নতুন কাজ আনো |
| `git merge upstream/main` | Fork-এর `main` হালনাগাদ |
| `git fetch origin pull/42/head:pr-42` | PR #42 নিজের কম্পিউটারে চালিয়ে দেখা |
| `gh pr create` | Terminal থেকে PR |
| `gh pr checkout 42` | PR নামানো, এক লাইনে |

PR বিবরণে `Closes #7` লিখলে merge হলেই issue বন্ধ। push `origin`-এ, fetch `upstream` থেকে।

## প্রতিদিনের ছন্দ (GitHub Flow) — অধ্যায় ২৪

**git pull** → **switch -c** → **commit** → **push** → **PR** → **review** → **merge** → **deploy**

`main` সবসময় চালু অবস্থায়। Branch-এর আয়ু ১–৩ দিন। একটা PR = একটা কাজ।



Licensed to sakiron@gmail.com · Do not redistribute

# কোন ভুলে কোন command • সাধারণ error

আগে থামুন, `git status`, বার্তা পুরো পড়ুন

## ভুল শোধরানোর ছক

অধ্যায় ৯–১২

| | |
|---|---|
| **Commit করিনি** | |
| ফাইলে লেখা বাদ দিতে চাই | `git restore ফাইল` ⚠ |
| ভুল ফাইল add করেছি | `git restore --staged ফাইল` |
| ভুলে ফাইল মুছে ফেলেছি | `git restore ফাইল` |
| অর্ধেক কাজ, জরুরি অন্য কাজ | `git stash push -u -m "..."` |
| **Commit করেছি, push করিনি** | |
| শেষ commit-এ ফাইল বাদ / message ভুল | `git commit --amend` |
| Commit ভাঙতে চাই, কাজ রেখে | `git reset --soft HEAD~1` |
| ভুল branch-এ ২টা commit | `git branch নতুন`<br>`git reset --hard HEAD~2` ⚠ |
| কাজটাই ফেলে দিতে চাই | `git reset --hard HEAD~1` ⚠ |
| **Push করে ফেলেছি** | |
| একটা commit বাতিল | `git revert <hash>` |
| পুরো merge বাতিল | `git revert -m 1 <hash>` |
| **কিছু খুঁজে পাচ্ছি না** | |
| Reset, rebase, branch মোছার পরে | `git reflog` |
| Detached HEAD-এ কাজ করেছি | `git switch -c নতুন-নাম` |

## .gitignore-এর মূল নিয়ম

অধ্যায় ৮

| | |
|---|---|
| `.env` | গোপন তথ্য — কখনো commit না |
| `venv/ node_modules/` | ফোল্ডার, আবার install করা যায় |
| `*.log __pycache__/` | নিজে তৈরি হওয়া ফাইল |
| `/build/` | শুধু মূল ফোল্ডারেরটা |
| `logs/* !logs/keep.txt` | ফোল্ডারের সব, একটা বাদে |
| `git check-ignore -v ফাইল` | কোন নিয়মে আটকাচ্ছে |
| `git rm --cached ফাইল` | আগে commit হওয়া ফাইল track বন্ধ |

Lock ফাইল (`package-lock.json`, `poetry.lock`) commit করবেন। Secret একবার push হলে আগে key বদলান।

## সাধারণ error → সমাধান

অধ্যায় ২৯

| | |
|---|---|
| `! [rejected] ... (fetch first)` | অন্যের নতুন কাজ আছে → `git pull` তারপর `git push`। Force না। |
| `have diverged / Need to specify how to reconcile` | `git config --global pull.rebase true` → `git pull` |
| `local changes ... would be overwritten` | আগে `commit` বা `stash`, তারপর pull/switch |
| `untracked working tree files would be overwritten` | ফাইলটা সরান বা commit করুন |
| `refusing to merge unrelated histories` | `git pull --no-rebase origin main --allow-unrelated-histories` |
| `Permission denied (publickey)` | `ssh -T git@github.com` দিয়ে key পরীক্ষা; GitHub-এ `.pub` দিয়েছেন? |
| `has no upstream branch` | `git push -u origin নাম` |
| `HEAD detached at ...` | শুধু দেখছিলেন: `git switch main`। কাজ রাখতে: `git switch -c নাম` |
| `CONFLICT (content)` | Marker মিটিয়ে `git add` → `commit` বা `rebase --continue`। ভয় পেলে `--abort` |
| `not a git repository` | `pwd` দেখে প্রজেক্টের ফোল্ডারে `cd` |
| `Author identity unknown` | উপরের `user.name`, `user.email` setup |
| `LF will be replaced by CRLF` | শুধু সতর্কবার্তা; `core.autocrlf` ঠিক রাখুন |
| Vim-এ আটকে গেছি | `Esc` তারপর `:wq` (সেভ) বা `:q!` (বাতিল) |

## দুটো কাজের alias

| | |
|---|---|
| `git config --global alias.lg "log --oneline --graph --all --decorate"` | এরপর শুধু `git lg` |
| `git config --global alias.st "status -sb"` | এরপর শুধু `git st` |

**এই পাতার প্রতিটা লাইনের পেছনে একটা গল্প আছে।**
কেন হয়, কীভাবে এড়াবেন, আসল output সহ — ২৯১ পাতার বইয়ে।

বইটি দেখুন →


Licensed to sakiron@gmail.com · Do not redistribute




চিট শিট মনে করিয়ে দেয় · বই শেখায়

# Git ও GitHub: আদি থেকে অন্ত

প্রথম commit থেকে team-এ কাজ, GitHub Actions আর server-এ auto-deploy পর্যন্ত — একটা দোকানের ওয়েবসাইট বানাতে বানাতে, সহজ বাংলায়।

- ✔ ২৯ অধ্যায়, ৭ পার্ট, ২৯১ পাতা
- ✔ প্রতিটা command চালিয়ে মেলানো আসল output
- ✔ Conflict, rebase আর reflog দিয়ে কাজ উদ্ধার
- ✔ Pull Request, code review, team workflow
- ✔ GitHub Actions দিয়ে VPS-এ auto-deploy প্রজেক্ট
- ✔ AI coding agent-এর সাথে নিরাপদে Git
- ✔ প্রতিটা অধ্যায় শেষে সাধারণ ভুল আর অনুশীলন
- ✔ ১৮টা সাধারণ সমস্যার খোঁজার মতো সমাধান

**বইটি সংগ্রহ করুন →**

iftesamulohy.com/products/git-o-github

Licensed to sakilorb@gmail.com · Do not redistribute